VPU Module Description

1. Fetch Stage

Clock and Reset Signals

- clk_i: Input clock signal used to drive the sequential logic inside the module.
- rst_ni: Input reset signal, typically active low. It is used to initialize or reset the internal registers and logic.

Inputs from the CPU

- fetch_req_i: Request signal received from the CPU or the previous stage, indicating that an instruction needs to be processed by the fetch stage.
- insn_i: Input instruction data, i.e., the instruction code to be read by the fetch stage.
- cs_data_i: Input control/status data, which may contain additional information required for instruction execution.
- ls_addr_i: Input load/store address used when the instruction involves data memory access.
- inv_insn_i: Signal indicating whether the input instruction is invalid.

Pipeline Control Signals

- fetch_decode_req_o: Request signal sent to the decode stage, indicating that the fetch stage has data ready to be forwarded.
- decode_fetch_gnt_i: Grant signal received from the decode stage, indicating whether the decode stage is ready to accept new data.

Decoder-Related Outputs

- insn_o: Instruction data forwarded to the decode stage.
- cs_data_o: Control/status data forwarded to the decode stage.
- ls_addr_o: Load/store address forwarded to the decode stage.

Outputs to the CPU

- fetch_rvalid_o: Indicates whether the current stage output is valid.
- fetch_rcs_data_o: Control/status feedback returned to the CPU, potentially including error codes or other status information.

Internal Data and Control Registers

- insn_q, insn_d: Registers for instruction data. insn_d is the next-state value and insn_q is the current-state value.
- cs_data_q, cs_data_d: Registers for control/status data.
- ls_addr_q, ls_addr_d: Registers for the load/store address.
- rcs_data_q, rcs_data_d: Registers for feedback control/status data returned to the CPU.
- rvalid_q, rvalid_d: Registers indicating data validity.

Pipeline Control Logic Signals

- fetch_decode_req_d, fetch_decode_req_q: Signals used to control whether a request should be issued to the decode stage.
- fetch_gnt: Locally generated grant signal based on decode_fetch_gnt_i and internal conditions.

- fetch_enable: Signal controlling whether the fetch stage is active.
- fetch_decode_enable: Indicates whether the fetch stage is allowed to send data to the decode stage.

Functionality

- The fetch module is responsible for obtaining instructions and related data from the CPU and managing pipeline flow control.
- fetch_gnt determines whether a transfer request from the fetch stage to the decode stage is granted.
- fetch_enable becomes active when both fetch_req_i and fetch_gnt are asserted, enabling instruction fetch and transfer.
- Internal registers such as insn_q, cs_data_q, and ls_addr_q buffer the current-cycle data for use in the next cycle, ensuring continuity and correctness.
- fetch_decode_req_d and fetch_decode_req_q manage request state so that instructions are correctly issued to the decode stage after a grant handshake.
- When rst_ni is asserted, all outputs and internal state registers are cleared to guarantee a known initial state.
- When the fetch stage receives a fetch request and a grant from the decode stage, it becomes active and sends a request to the decode stage.
- If a valid instruction is available, the fetch stage forwards the instruction and related data to the next stage and asserts the corresponding valid signal.
- At every clock cycle, internal state and outputs are updated according to the enable and grant conditions.

Summary

- In summary, the fetch module handles instruction acquisition in the pipeline. It receives instructions, manages the flow-control handshake with the decode stage, and forwards instruction/state information to both the next pipeline stage and the CPU. This module forms the basis for efficient and orderly pipeline operation.

2. Decoder

Input Signals

- clk_i: Clock signal.
- rst_ni: Reset signal.
- insn_i: 32-bit input instruction.
- cs_data_i: Control and status data.
- ls_addr_i: Load/store address.

Output Signals

- decode_fetch_gnt_o: Grant signal sent back to the fetch stage.
- decode_execute_valid_o: Signal indicating that decoding is valid.
- inv_insn_o: Invalid instruction indicator.

Operand Outputs

- vs1_o, vs2_o: Source vector register addresses.
- immediate_o: Immediate value.
- vd_o: Destination vector register address.

Configuration Outputs

- sew_o: SEW mode (element width).
- vstart_o, vl_o: Start index and length for vector operations.
- vm_o: Mask enable.
- mem_mode_o: Memory access mode.
- alu_mode_o: ALU operation mode.
- mul_mode_o: Multiplier operation mode.

Functionality

- The decoder classifies and parses the input 32-bit instruction, recognizes the opcode, and categorizes the instruction.
- Based on the opcode, it determines whether the instruction is a vector operation (OP-V), floating-point load (LOAD_FP), or floating-point store (STORE_FP).
- It extracts function fields (funct3, funct6) and register addresses (vs1, vs2, vd), then uses them to configure the exact ALU and multiplier operation modes, such as addition or multiplication.
- It generates the immediate value and other control signals, such as enable signals and the vector mask enable (vm).
- Case statements are used to match opcodes and function fields to determine the specific operation type.
- It checks instruction legality by validating opcode/function-field combinations and sets inv_insn_o if the instruction is illegal.
- It configures ALU, multiplication, and memory modes according to the decoded instruction type.
- It receives requests from the fetch stage, sends grant signals back, and forwards the decode results to the execute stage.

3. Execute Stage

Parameters

- VLEN: Vector length, i.e., the total data width.
- ELEN: Maximum element width.
- LANES: Number of parallel processing lanes.
- Additional parameters define data-path widths and iteration-related widths.

Inputs

- clk_i and rst_ni: Clock and reset.
- vd_i: Destination register index used to identify which vector register receives the result.
- rdata_a_i, rdata_b_i, rdata_c_i: Three register-data input buses providing source operands for arithmetic or logic operations.
- immediate_i: Immediate value used in immediate-based operations.
- sew_i: Scalar element width, defining the width of each processed data element.
- alu_cin_i: Carry-in for the ALU.
- alu_mode_i: ALU operation mode, such as add, subtract, or bitwise logic.
- mul_mode_i: Multiplier operation mode.
- ls_enable_i: Load/store enable signal.
- mem_mode_i: Memory operation mode, including access type and behavior.
- load_data_i: Data loaded from memory or another data source.

Outputs

- we_a_o: Register write-enable signal.
- wdata_a_o: Data to be written back to the destination register.
- store_data_o: Data to be written to memory.
- lane_done_o: Per-lane completion signals.

Debug Signals

- dbg_alu_carry_o: Displays the carry result of the ALU operation.
- dbg_arith_result_o: Displays the arithmetic result generated by the ALU.
- dbg_arith_valid_o: Indicates whether the arithmetic result is valid.
- dbg_iteration_o: Iteration counter, useful for debugging or performance monitoring.

Data Distribution and Preprocessing

- In the always_comb block, the module slices rdata_a_i and rdata_c_i and distributes them to each processing lane. At the same time, it assembles lane results into the write-back buffer wdata_a_d.

Preprocessing for Multiply Reduction Sum

- When mul_mode_i.redsum is enabled, only lane 0 loads the actual input from rdata_b_i, while the remaining lanes are cleared to zero. This prepares the datapath for a reduction-sum accumulation. If reduction is not enabled, each lane directly receives its corresponding slice of rdata_b_i.

Registering the Write-Back Buffer

- On each rising clock edge, wdata_a_q is either cleared by reset or updated from wdata_a_d, ensuring that data is available in the next cycle.

Final Write-Back Output

- If mul_mode_i.redsum is enabled, only the final accumulated result is output; otherwise, the full buffered vector result wdata_a_f is written to wdata_a_o.

Lane-Done Signal Update

- lane_done_d is updated from lane_done on every clock edge, unless reset is active. lane_done_o then outputs the registered completion state in the following cycle, providing synchronized lane-completion information to external modules.

Write-Enable Handling

- we_a_d collects write-enable requests from all lanes using a bitwise OR reduction. If any lane requests a write, we_a_d is set high. In the next cycle, this value is passed to we_a_o.

Store-Data Management

- store_data_q buffers store data on the clock edge. If reduction-sum mode is enabled, the data from all lanes is accumulated lane by lane; otherwise, the lane data is passed through directly. The final processed data is driven onto store_data_o.

ALU and Load-Data Processing

- The module slices alu_cin_i and load_data_i lane by lane, producing lane_alu_cin and lane_load_data, and collects per-lane store results into store_data_d.

4. Lanes

Module Parameters

- LLEN: Vector length per lane in bits (assumed to be 128 bits here).
- ELEN: Maximum element width in bits (32 bits in this design).
- ITERATIONS: Number of iterations, derived from vector length divided by element width.
- ELEN_B and LLEN_B: Byte-based widths of the element and lane vector length.
- BW_I: Bit width used to index the number of iterations, computed using $clog2().

Inputs

- clk_i and rst_ni: Clock and reset.
- Control signals and mode fields such as sew_mode_e and alu_mode_s, used to control arithmetic and multiplication behavior.
- Source vector register addresses vs1_i, vs2_i, vs3_i and destination register address vd_i.
- byte_enable_i and alu_cin_i: Byte-enable controls and ALU carry-in inputs.

Outputs

- done_o: Operation-complete signal.
- store_data_o: Data output for store operations.
- dbg_* signals: Debug information such as arithmetic results and iteration count.

Main Submodules

- vpu_register_file: Register file used to store and provide vector data.
- vpu_arith_unit: Arithmetic unit that performs arithmetic and multiplication operations.
- vpu_lane_controller: Controller that manages lane operation, data flow, execution, and state.

Functionality and Workflow

- The register file reads vector data according to the input addresses.
- The controller receives register data and processes it according to the configured arithmetic or multiplication mode.
- The result may be an arithmetic result or load/store data.
- Processed results are written back to the destination vector register when required, or forwarded to the memory subsystem.
- Multiple debug outputs are provided to monitor arithmetic results, valid status, write enable, and related execution states.

5. Lane Controller

Input Signals

- clk_i: Clock input used to synchronize the module.
- rst_ni: Active-low reset used to initialize or clear internal state.
- alu_mode_i: ALU mode input specifying operations such as add, subtract, or logic functions.
- mul_mode_i: Multiplier mode input, including optional behaviors such as multiplication and reduction sum.
- sew_i: Scalar element width mode, which determines operation granularity.
- immediate_i: Immediate input used together with register data during execution.

- alu_cin_i: Carry-in input for the ALU.
- rf_data_a_i, rf_data_b_i, rf_data_c_i: Register file data inputs used as source operands.
- arith_valid_i: Indicates that the arithmetic result is complete and valid.
- arith_result_i: Arithmetic result input.
- ls_enable_i: Load/store enable signal.
- ls_op_i: Load/store operation type.
- load_data_i: Load-data input.

Output Signals

- op_a_o, op_b_o, op_c_o: Operand outputs for arithmetic or logic operations.
- we_o: Write-enable output controlling write-back to the register file or memory.
- wdata_o: Write data output.
- done_o: Signal indicating completion of the current lane operation.
- dbg_iteration_o: Debug iteration counter showing the currently processed vector-element index.

Internal Control Signals

- arith_enable: Indicates whether an arithmetic operation is enabled based on alu_mode_i.enable or mul_mode_i.enable.
- next_iteration: Indicates whether execution should proceed to the next iteration. It is asserted when the current arithmetic result is valid and arithmetic processing is enabled.
- last_iteration: Indicates whether the iteration counter has reached the final step.
- immediate: Internal logic vector that stores the immediate value after extension according to sew_i.
- slice_a, slice_b, slice_c: Sliced data fields extracted from the register-file data for the current iteration.
- iteration_q, iteration_d: Current and next values of the iteration counter.
- arith_data_q, arith_data_d: Registers that accumulate arithmetic result data across iterations.
- we_d, wdata_d: Temporary registers for write-enable and write-data decisions.

Decision Logic

- op_a, op_b, op_c select the actual operands according to the instruction type, such as reverse subtraction or immediate-based computation.
- we and wdata determine whether a write occurs and what data is written, based on the operation type and current execution state.
- done indicates completion of the current lane process, based on memory-operation type and write behavior.

Key always Blocks

- Iteration Control (always_comb): Computes iteration_d. If next_iteration is asserted, iteration_d increments modulo ITERATIONS; otherwise it remains unchanged.
- Iteration Counter (always_ff): Updates iteration_q on the rising clock edge, or resets it to zero when rst_ni is deasserted.
- Extend Immediate (always_comb): Extends the immediate according to sew_i so that it matches the current data width.
- Operand Inputs (always_comb): Selects the data slice corresponding to the current iteration as the operand source.
- Reverse Subtraction Logic (always_comb): Adjusts op_a and op_b for special cases such as reverse subtraction.
- Carry-In Logic (always_comb): Determines op_c based on alu_cin_i or other operation requirements.

- Combine Arithmetic Data (always_comb): Combines per-iteration arithmetic results into a complete vector result.
- Update Arithmetic Data (always_ff): Registers the accumulated arithmetic result.
- Update Write Enable and Data (always_ff): Registers the final write-enable and write-data decisions.
- Write Data Logic (always_comb): Decides whether a write should occur and what data should be written.

Summary

- Each always block implements a critical part of the vpu_lane_controller module, including initialization, state progression, operand preparation, arithmetic-data combination, and write-control generation. Together, these blocks ensure that the lane controller behaves as intended.

6. MEM / Writeback Stage

Input Signals

- clk_i: Clock input controlling sequential behavior.
- rst_ni: Reset signal used to initialize or reset the module.
- lane_done_i: Completion signals from all lanes, indicating whether lane operations have finished.
- byte_enable_i: Byte-enable signal used to specify which data bytes are valid.
- ex_wb_req_i: Request from the execute stage to the writeback stage.
- vs1_i, vs2_i, vd_i: Source and destination register indices.
- vm_i: Mask register used for conditional execution.
- immediate_i: Immediate value used in certain immediate operations.
- sew_i: Scalar element width indicating the width of processed data.
- vstart_i, vl_i: Vector start position and vector length.
- alu_mode_i, mul_mode_i, mem_mode_i: Mode fields for ALU, multiplier, and memory access.
- ls_req_i: Load/store request.
- store_enable_i: Store enable signal.
- store_data_i: Data to be stored.
- data_gnt_i, data_rdata_i, data_rvalid_i: Data grant, returned read data, and read-valid signals.

Output Signals

- ex_wb_gnt_o: Grant output from writeback to the execute stage.
- vs1_o, vs2_o, vd_o: Register indices forwarded to the next stage.
- vm_o: Forwarded mask register output.
- immediate_o: Forwarded immediate value.
- sew_o: Forwarded scalar element width.
- vstart_o, vl_o: Forwarded vector start and vector length.
- alu_mode_o, mul_mode_o, mem_mode_o: Forwarded operation mode outputs.
- lanes_done_o: Combined completion signal for all lanes.
- ls_gnt_o, ls_valid_o: Load/store grant and valid outputs.
- load_data_o: Loaded data output.
- data_req_o, data_addr_o, data_we_o, data_be_o, data_wdata_o: Memory-interface outputs including request, address, write enable, byte enable, and write data.

Debug Signals

- dbg_insn_complete_o: Indicates whether the instruction has completed in the writeback stage.

- dbg_ex_wb_hs_o: Handshake indicator between execute and writeback stages.
- dbg_ex_wb_en_o: Indicates whether the writeback stage is enabled.
- dbg_byte_enable_o: Debug output showing which bytes are currently enabled.

Condition and State Update Logic

- A combinational block updates alu_mode_d, mul_mode_d, and mem_mode_d. If the execute/writeback handshake ex_wb_hs is asserted, the modes are loaded from the input signals; otherwise, if ex_wb_en_d is deasserted, the mode-enable bits are cleared.

Sequential State Transfer

- A clocked always_ff block updates internal state and output registers on each cycle. During reset, all relevant registers are initialized to known values. During normal operation, they are updated from the corresponding _d next-state signals, ensuring that the latest control/data information is transferred correctly.

7. Vector Register File

Module Parameters and Ports

- LLEN defines the full length of each vector register and is set to 128 bits in this design.
- Ports include clk_i and rst_ni, three read ports (raddr_a_i, raddr_b_i, raddr_c_i and their outputs rdata_a_o, rdata_b_o, rdata_c_o), and one write port (waddr_a_i, wdata_a_i, wbe_a_i, and we_a_i).

Memory Storage

- The storage array mem is implemented as a two-dimensional array. mem[NUM_REGISTER] represents 32 registers (based on a 5-bit address width), and each register consists of NUM_BYTES bytes, where each byte is 8 bits and NUM_BYTES equals LLEN/8.

Read Operation

- The register file uses combinational logic for asynchronous reads. Therefore, rdata_a_o, rdata_b_o, and rdata_c_o directly reflect the value stored at the selected address without waiting for a clock edge.

Write Operation

- The write path is synchronous. On the rising clock edge, if reset is inactive and we_a_i is high, a write is performed.
- Byte-enable support through wbe_a_i allows byte-level selective updates. Each bit in wbe_a_i controls whether the corresponding byte should be written.

Functional Flow

- When we_a_i is asserted at the positive clock edge and reset is inactive, the module updates the selected bytes of the target register according to wbe_a_i.
- Read operations directly output the data stored at the selected addresses through asynchronous access, providing low-latency data retrieval.

Key Design Features

- Byte-level write enable allows partial vector updates instead of rewriting the whole vector register, improving flexibility and efficiency.
- Asynchronous read access optimizes read latency and is well suited to high-performance computing applications requiring fast data access.

8. Execution Controller

Core Role

- The execution controller is the central control block for the execute stage of the vector processing unit, particularly for memory accesses and ALU-related operations.
- Its main responsibilities are managing memory-interface signals, controlling byte-enable information on the data path, and handling ALU carry-in signals.

Memory Access Control

- According to mem_mode_i, the module decides whether to issue load or store operations.
- It uses a state machine to manage load/store requests, ensuring that request signals (ls_request_o) are issued at the correct time and that state is updated once a response is received.
- The state machine typically uses three states—idle, hold, and response—to control the memory operation lifecycle.

Data Masking and Byte Enable

- Based on vstart_i, vl_i, and the vector mask register value, the module computes the effective operation mask.
- This mask is then mapped to byte_enable_o and alu_cin_o, enabling fine-grained control of both the memory data path and ALU carry propagation.

Store-Data Buffering and Forwarding

- During store operations, store_data_i is buffered until the store can be safely issued, ensuring data consistency and correctness across multi-cycle behavior.

Special Execution Control

- For multiply-accumulate instructions (mac_insn_i), the module specially configures the destination/source register path via vs3_o so that data is stored or updated correctly.

Overall Summary

- Overall, vpu_ex_controller acts as the control center of the execute stage, coordinating data flow, memory operations, and ALU processing. This strategy improves execution efficiency while also increasing flexibility and responsiveness.

Input Signals

- clk_i: Standard clock input for sequential logic.
- rst_ni: Active-low reset input.
- vd_i: Destination vector register index.
- mac_insn_i: Flag indicating whether the current instruction is a multiply-accumulate instruction.
- mem_mode_i: Memory-mode structure containing information such as operation enable and load/store type.
- alu_mode_i: ALU-mode structure containing details of the arithmetic operation.
- sew_i: Scalar element width.
- vstart_i: Start index for the vector operation.
- vl_i: Vector length.
- vm_i: Vector mask used for conditional execution.
- lanes_done_i: Indicates whether all lanes have completed the current operation.
- store_data_i: Input store data.

- ls_grant_i: Memory grant signal.
- ls_valid_i: Indicates that memory data is valid.

Output Signals

- vs3_o: Source vector register index used for store operations.
- alu_cin_o: ALU carry-in output.
- byte_enable_o: Byte-enable output for memory operations.
- ls_request_o: Memory request signal.
- store_enable_o: Store enable output.
- store_data_o: Store data output.
- ls_enable_o: Load/store enable output.

Internal Logic Signals

- ls_state_q, ls_state_d: Registers for the load/store state machine.
- ls_req: Internal load/store request signal.
- load_insn, store_insn: Flags indicating whether the current instruction is a load or store.
- mem_insn: Flag indicating whether the current instruction is a memory operation.
- mem_op: Memory operation type such as load or store.
- store_data_d, store_data_q: Registers used to buffer store data.
- mask, mask_register_d, mask_register_q: Masking-related registers and logic used to generate and maintain the effective operation mask.

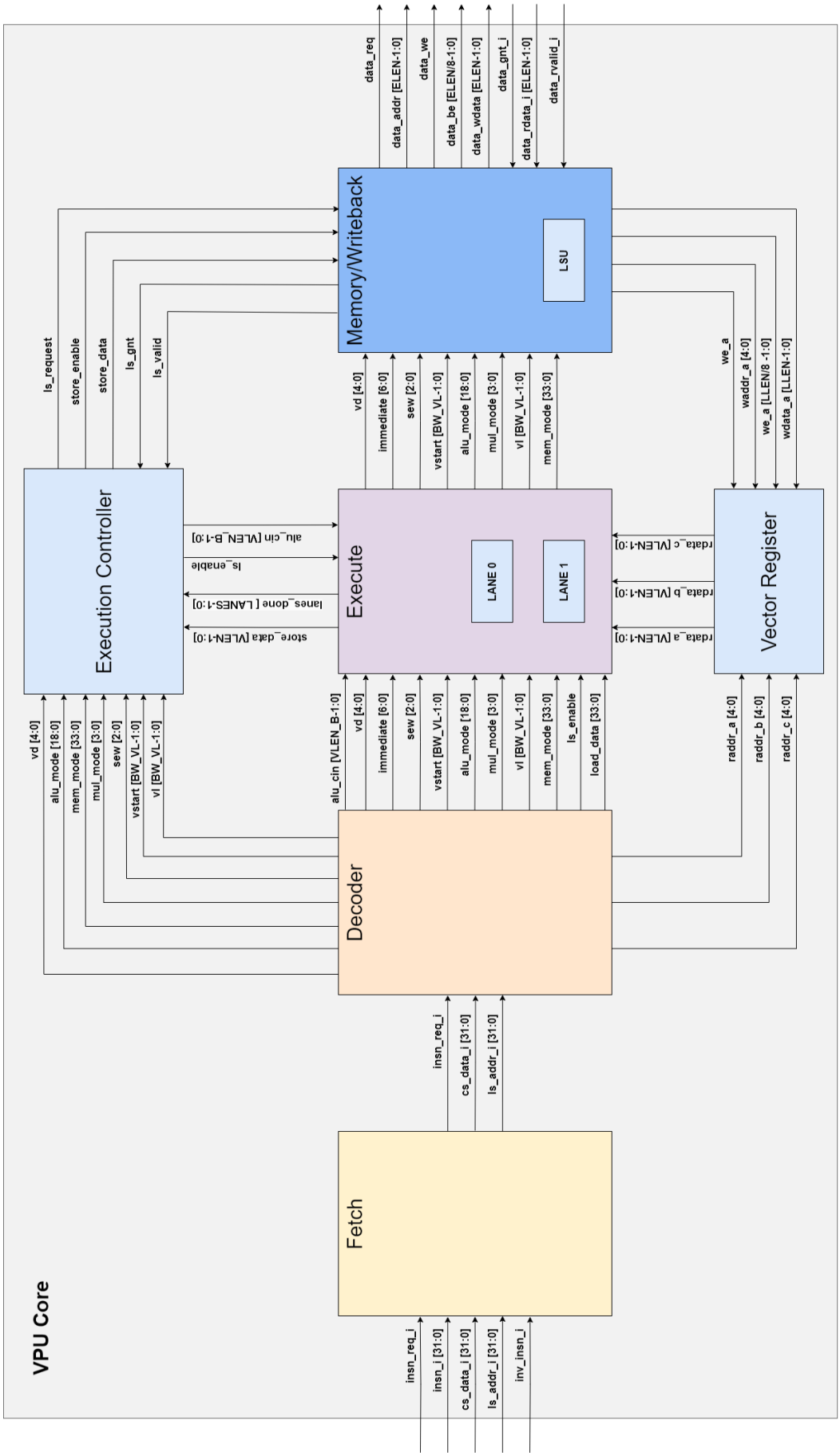
Key always Blocks

- Memory-Access State Machine (always_comb): Determines ls_state_d and ls_req according to the current state and input conditions.
- Store-Data Handling (always_comb): Buffers store_data_i into store_data_d when a store instruction is accepted in the idle state.
- State and Store-Data Update (always_ff): Updates ls_state_q and store_data_q on the clock edge, or resets them in reset.
- Mask-Register Update (always_ff): Updates mask_register_q from mask_register_d on each clock edge.
- Mask Mapping to Byte Enable (always_comb): Maps the effective mask to byte_enable and alu_cin according to the selected SEW mode.
- Active-Mask Computation (always_comb): Computes the mask body from vstart_i and vl_i and combines it with the stored mask state to derive the active mask used for execution.

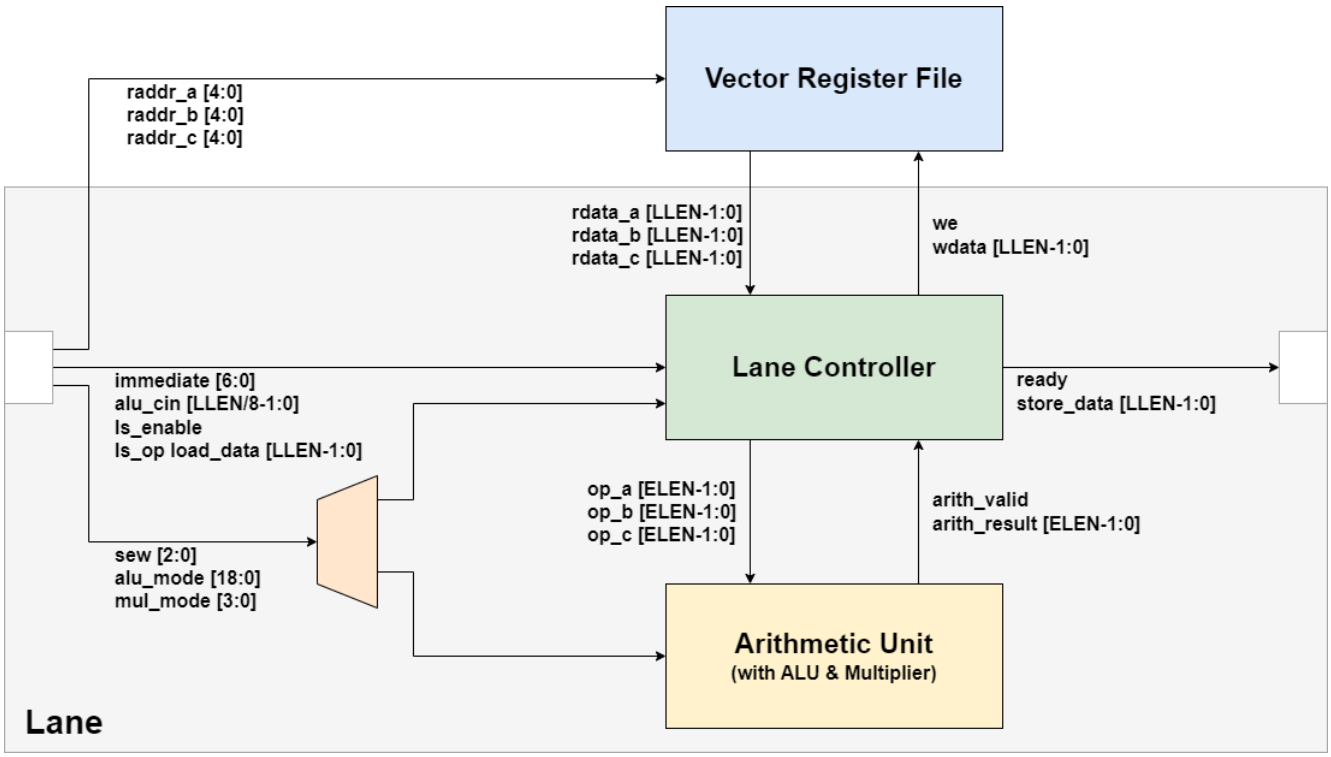
Summary

- These blocks collectively maintain the load/store state machine, buffer store data, generate effective byte-enable and carry-in signals, and preserve mask state across cycles, which are all essential to correct execution control.

Architectural Overview



Lane


Vector Register File
raddr_a [4:0]
raddr_b [4:0]
raddr_c [4:0]
rdata_a [LLEN-1:0]
rdata_b [LLEN-1:0]
rdata_c [LLEN-1:0]
we
wdata [LLEN-1:0]
Lane Controller
immediate [6:0]
alu_cin [LLEN/8-1:0]
ls_enable
ls_op load_data [LLEN-1:0]
ready
store_data [LLEN-1:0]
op_a [ELEN-1:0]
op_b [ELEN-1:0]
op_c [ELEN-1:0]
arith_valid
arith_result [ELEN-1:0]
sew [2:0]
alu_mode [18:0]
mul_mode [3:0]
Arithmetic Unit
(with ALU & Multiplier)
Lane